

GLIMMER-CS-EASY-01:C语言入门

part1

1.(1)GCC是C语言最常用的编译器之一，可以将高级语言代码转化成机器码，编译流程如下：

预处理（生成.i文件）--编译（将代码转化成汇编语言，生成.s文件）--汇编（将汇编语言转化成机器指令，生成.o文件）--链接（合并目标文件和系统库，生成可执行文件）

(2)MinGW是一个开发工具集（翻译一下就是：windows下，极简的GCC及相关工具集），本身不是编译器，但是能使得GCC在Windows上顺利运行。

该工具集包括但不限于编译器，汇编器，链接器，调试器。

2. (1) c_cpp_properties.json 的作用

服务于编辑体验：

- 代码自动补全
- 静态错误检查（这种检查是在编辑同时进行的，不需要编译）
- 头文件识别（通过配置includePath，使得编辑器能够找到头文件）
- 快速跳转和悬浮提示（跳转定义：ctrl+函数名，跳转函数定义；悬浮提示：通过鼠标悬浮查看变量或函数的类型和注释说明）

(2) launch.json 的作用

- 指定待调试的可执行文件路径（program）
- 调用调试器与后端绑定

通过type和minDebuggerPath字段，声明调用当前文件所使用的调试器类型及其物理路径（其实就是绝对路径），使编辑器能够接受并解析返回参数（包括但不限于内存状态，寄存器数据什么的。太复杂了。。）

- 控制调试器即控制程序启动时的初始状态）

(3) tasks.json 的作用

- 编译任务自动化*（太拗口了。。）：

调用GCC（或其他编译器）及编译参数（例如-o参数指定输出的可执行文件路径），将源代码编译成可执行文件。

3.编辑器中C语言插件的作用

- 使编辑器真正理解C语言代码

VS code本质是一个文本编辑器，没有理解C语言的能力，安装插件是他具备了识别C语言，自动补全补全，提示（高亮，参数，错误）等等功能。

- 传递指令和编译结果

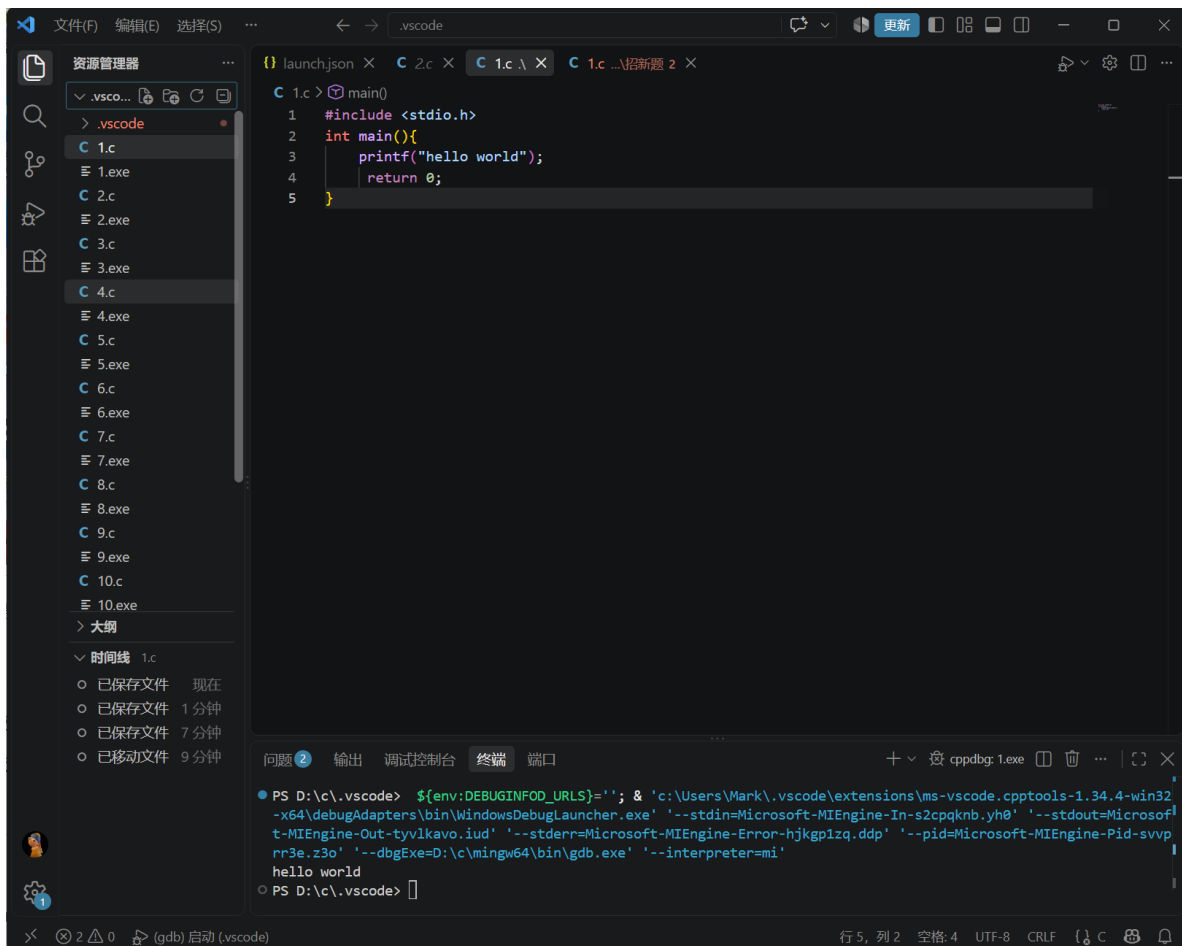
插件能将“调试”（“运行”）指令传递给GCC，将“断点”（“单步执行”）指令传递给GDB [指令还有很多，不便枚举]，并将编译结果通过编辑器反馈

4.关于launch.json

其基本配置如图：

```
{
  // 使用 IntelliSense 了解相关属性。
  // 悬停以查看现有属性的描述。
  // 欲了解更多信息，请访问: https://go.microsoft.com/fwlink/?linkid=830387  "version": "0.2.0",
  "configurations": [
    {
      "name": "gcc.exe - 生成和调试活动文件", // 该调试任务的名字，启动调试时会在待选列表中显示
      "type": "cppdbg",
      "request": "launch",
      "program": "${fileDirname}\\${fileBasenameNoExtension}.exe", "args": [],
      "stopAtEntry": false, //程序启动后一直运行，直至断点。如果是true，就会在main函数第一行自动暂停
      "cwd": "${workspaceFolder}",
      "environment": [],
      "externalConsole": false, //声明程序在内置终端（也就是底部小窗）运行；反之，在外置控制台运行
      "MIMode": "gdb",
      "miDebuggerPath": "C:\\mingw64\\bin\\gdb.exe", //声明当前文件的绝对路径
      "setupCommands": [
        {
          "description": "为 gdb 启用整齐打印",
          "text": "-enable-pretty-printing",
          "ignoreFailures": true
        }
      ],
      "preLaunchTask": "C/C++: gcc.exe build active file" // 调试前的预执行任务，这里的值是tasks.json文件中对应的编译任务，也就是调试前需要先编译
    }
  ]
}
```

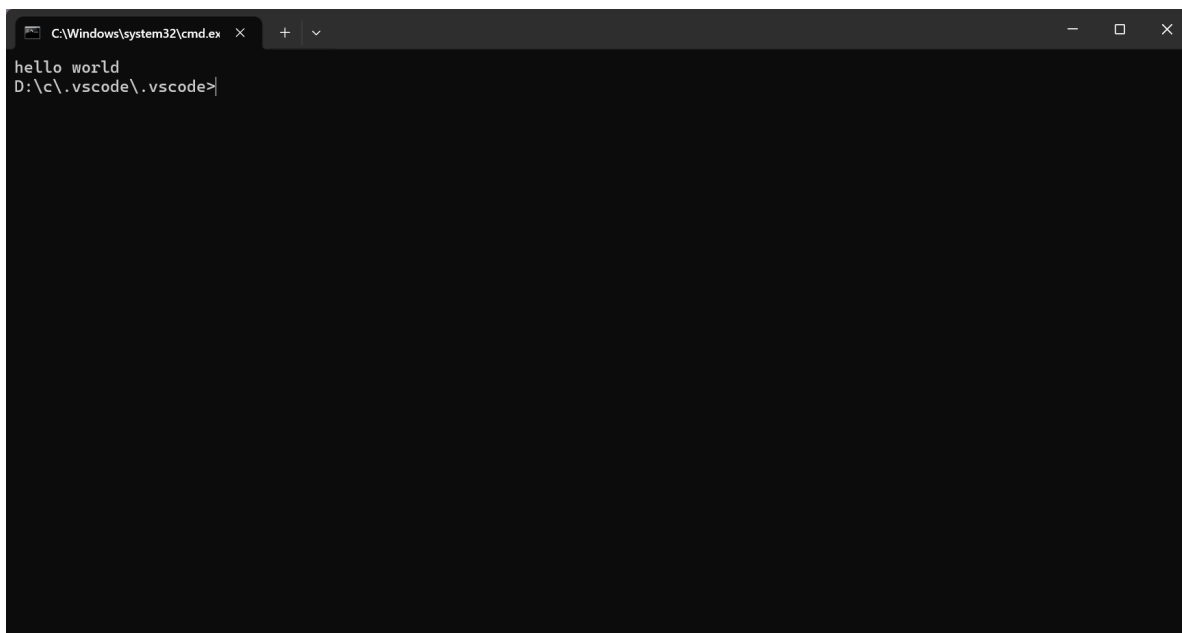
显然地，程序在内置终端输出一是一件非常自然的事：



当我在尝试更改为外置终端时，遇到了不小的挫折：

- 第一次 我仅仅只改变了externalConsole的状态，发现程序的输出位置没有发生改变，于是我思考：在launch.json内部一定有某个声明指向了目标文件
- 第二次 我想到了program，但是疑惑的是，在我更改为.exe的地址之后，系统却报错，这让我很是苦恼。因为我之前接触过Linux，惯性思维让我在单斜杠上花费了不知道多少时间。
- 第三次 在多次与AI沟通依然不能解决问题后，我在B站上找到了答案，才发现：原来program的绝对路径需要使用双反斜杠，且我的miDebuggerPath也并未修改。

幸运的是，最终我还是完成了任务：



由此我生发出了两点感慨：

一是，我的知识检索路径需要更新，先在网络上寻找，然后再询问AI（AI实在是给我蠢哭了）

二是，为什么launch.json这么复杂，难道每一次运行都需要复制一遍目标文件路径吗（如果想要用外置终端输出结果）？

AI说"`fileDirname{fileBasenameNoExtension}.exe`"可以运行当前文件，突然之间，我注意到题目中的launch.json，program同样也是"`fileDirname{fileBasenameNoExtension}.exe`"。于是，在我的尝试之下，外置终端带给了我两个消息，好消息是，这个声明是正确的，坏消息是，输出之后会闪退，当然了在return打断点就好啦（机智如我），后来查了AI，告诉我要写`getchar()`或者`system("pause")`（好吧，还是好麻烦）

part2

一、

1. 变量类型

- 什么是变量类型？

变量类型是对变量基本特征（如内存占用，表现形式）的刻画，他定义了数据的存储方式和操作规则。

- 变量类型的重要性

变量类型对数据的刻画保证了程序运行的规范（内存管理，操作规则）、稳定（代码运行时能通过检查数据类型来保证安全性）和高效（内存优化）

- *变量的选择

存放年龄，应该选择整型int (`int age = 18;`)

存放apple，当然不能选择char类型，char只有一个字节大小（这里站不下这么多大字母！），应当选择数组（`char word[] = "apple";`）每一个元素对应一个字节（即一个字母）

2. 数组起始于边界

- 数组的第一个元素下标通常是0（我听说程序员数数有从0开始的习惯，偷笑，还有就是，这题目有语病，哈哈[礼貌]）
- 当读取下标超出其有效范围属于越界（但是不会报错，可能连warning都没有），导致读取到数组之外的未知空间，可能是垃圾值，可能是空，可能是正常，还有可能读取到受保护的数据，这使得程序十分不稳定，很有可能崩溃（所以一般都要`index >= 0 && index < length`）。
- 这种错误的发生一般源于从1开始计数的习惯，以及对动态内存的粗糙处理。（当然我觉得编译器也需要负一定责任）

3. 流程控制 一循环结构（英文分号间隔太短了，使得文本有点看不清楚，我就全打成中文了）

说白了，for和while差别不大，都是只要满足循环条件就一直循环，只是for一般用于有限次循环（即已知次数），while一般用于无限次循环

- for循环：

```
for(初始化可以有; 条件判断; 迭代可以有){  
    循环体;  
}
```

下面以遍历数组举例：初始化`i = 0`；作用是将计数值`i`设置成0（设置迭代的起点，但是好像只有C99之后的版本才能在for里面初始化）条件判断`i < length`（即判断循环是否还能继续的标准，这里千万不能加`=`，不然就炸缸越界了），迭代`i++`（使循环的变量更新，相当于‘累积’，我记得迭代还可以写成这样`a[i++] = num`，算是利用副作用迭代了吧）

- while循环

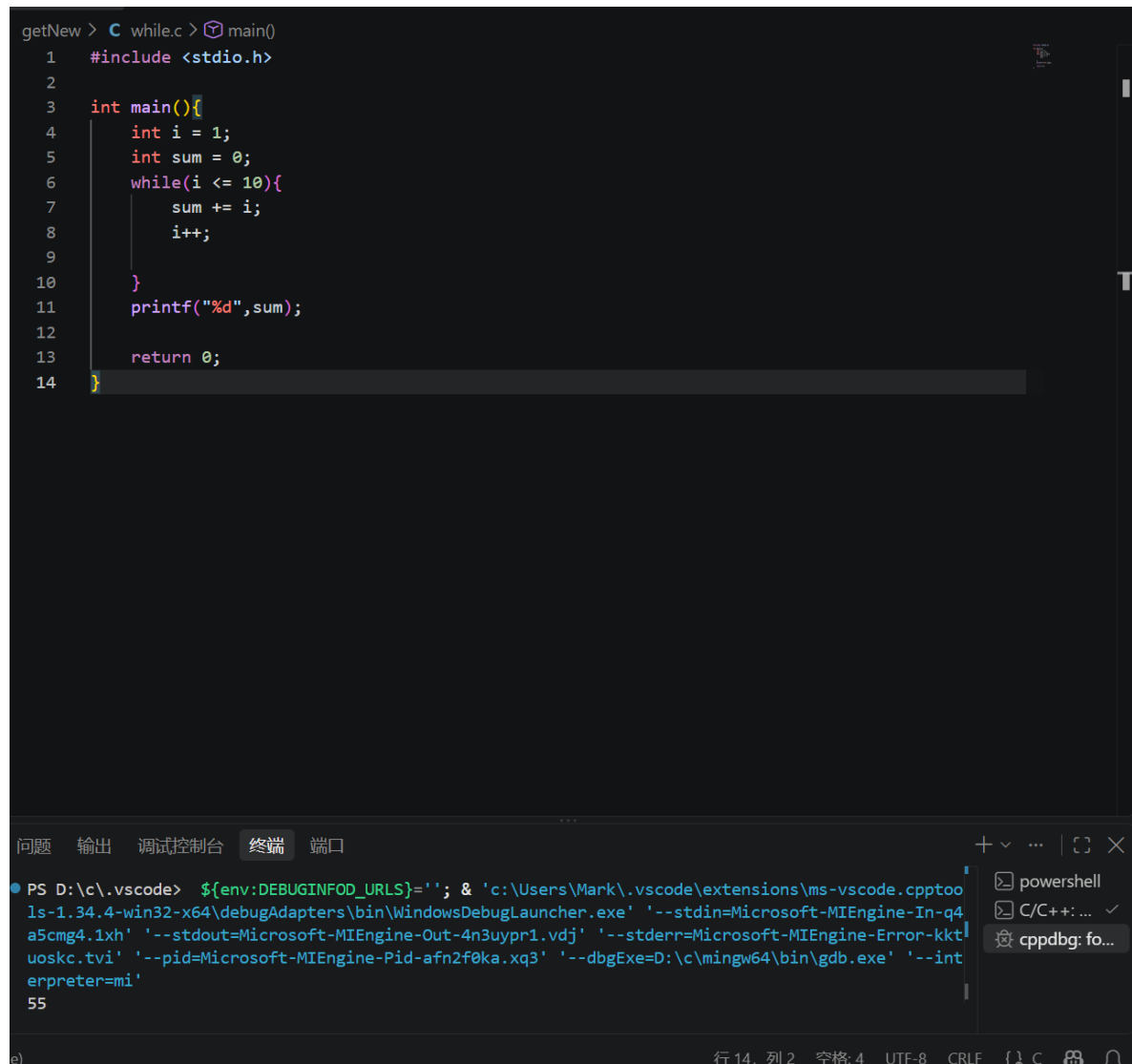
```
while(条件判断){  
    循环体;  
}
```

- 当然还有do while循环:

```
do{  
    循环体;  
}while (条件判断);
```

do while和while只有首次执行条件上有区别，do while是先执行一次循环再判断条件（just do it，有点先斩后奏那味儿），while是先进入条件判断再决定是否执行循环

代码截图如下



```
getNew > C while.c > main()  
1  #include <stdio.h>  
2  
3  int main(){  
4      int i = 1;  
5      int sum = 0;  
6      while(i <= 10){  
7          sum += i;  
8          i++;  
9      }  
10     printf("%d",sum);  
11  
12     return 0;  
13  
14 }
```

问题 输出 调试控制台 终端 端口

```
PS D:\c\.vscode> ${env:DEBUGINFOD_URLS}=''; & 'c:\Users\Mark\.vscode\extensions\ms-vscode.cpptools-1.34.4-win32-x64\debugAdapters\bin\WindowsDebugLauncher.exe' '--stdin=Microsoft-MIEngine-In-q4a5cmg4.1xh' '--stdout=Microsoft-MIEngine-Out-4n3uypr1.vdj' '--stderr=Microsoft-MIEngine-Error-kktuoskc.tvi' '--pid=Microsoft-MIEngine-Pid-afn2f0ka.xq3' '--dbgExe=D:\c\mingw64\bin\gdb.exe' '--interpreter=mi'  
55
```

行 14, 列 2 空格: 4 UTF-8 CRLF { } C

```
1  #include <stdio.h>
2
3  int main(){
4      int i = 1;
5      int sum = 0;
6      do{
7          sum += i;
8          i++;
9      }while(i<=10);
10     printf("%d",sum);
11     return 0;
12 }
```

问题 输出 调试控制台 终端 端口

```
PS D:\c\vscode> ${env:DEBUGINFOD_URLS}='' & 'c:\Users\Mark\.vscode\extensions\ms-vscode.cpptools-1.34.4-win32-x64\debugAdapters\bin\WindowsDebugLauncher.exe' '--stdin=Microsoft-MIEngine-In-q4a5cmg4.1xh' '--stdout=Microsoft-MIEngine-Out-4n3uypr1.vdj' '--stderr=Microsoft-MIEngine-Error-kktuoskc.tvi' '--pid=Microsoft-MIEngine-Pid-afn2f0ka.xq3' '--dbgExe=D:\c\mingw64\bin\gdb.exe' '--interpreter=mi'
55
```

powershell
C/C++: ...
cppdbg: fo...

行 2, 列 1 空格: 4 UTF-8 CRLF C

```
1  #include <stdio.h>
2
3  int main(){
4      int i = 1;
5      int sum = 0;
6      do{
7          sum += i;
8          i++;
9      }while(i<=10);
10     printf("%d",sum);
11     return 0;
12 }
```

问题 输出 调试控制台 终端 端口

```
PS D:\c\vscode> $(env:DEBUGINFOD_URLS='') & 'c:\Users\Mark\.vscode\extensions\ms-vscode.cpptools-1.34.4-win32-x64\debugAdapters\bin\WindowsDebugLauncher.exe' '--stdin=Microsoft-MIEngine-In-q4a5cmg4.1xh' '--stdout=Microsoft-MIEngine-Out-4n3uypr1.vdj' '--stderr=Microsoft-MIEngine-Error-kktuoskc.tvi' '--pid=Microsoft-MIEngine-Pid-afn2f0ka.xq3' '--dbgExe=D:\c\mingw64\bin\gdb.exe' '--interpreter=mi'
55
```

行 2, 列 1 空格: 4 UTF-8 CRLF

4. 流程控制—逻辑表达式

- 逻辑表达式 (如 `age > 18 && score >= 60`) 运算对象可以是任何类型的数据, 运算结果 (返回值) 是 0 (假) 或 1/非 0 (真) (C 语言的 `bool` 好像是后来补充的, 没有办法返回 `True` 或 `False`)
算术表达式 (如 `a+b*c`) 的运算对象是数值类型的数据 (指针也是, 但是指针的运算是针对地址而言的), 运算结果是较大的 (指精度高或者内存大) 数据类型 (比如 `int` 与 `double` 运算, 结果会是 `double`)
在此必须要做几点提醒
 - (1) 关于精度: `char/short < int < long < float < double` (诸如 `long long` 等就不在此列出)
 - (2) `int/int` 的结果会取整
 - (3) `double` 在计算时必然会产生误差 (`float` 就更不用说了), 所以能用整型就尽量用整型。
 - (4) `double` 能表示的范围是: 以 0 为对称点, 左右各一个连续区间 (具体是多少, 感觉意义不大); 0; 正无穷; 负无穷
- 逻辑运算符 `&&` 表示: 且 (我都要!)
逻辑运算符 `||` 表示: 或 (任意为真就行)
逻辑运算符 `!` 表示: 非 (比如不等于: `[! =]`)

```

1  #include <stdio.h>
2
3  ∨ int main(){
4      int a = 521;
5      int b = 6;
6      int c = 99;
7      int d = 114514;
8      int ret = (a+b<d-c&& c*a<d) || (d<a+b+c || a,b);
9      printf("%d",ret);
10 }

```

显然, $ret = 1$;

演示结果:

```

1  #include <stdio.h>
2
3  int main(){
4      int a = 521;
5      int b = 6;
6      int c = 99;
7      int d = 114514;
8      int ret = (a+b<d-c&& c*a<d) || (d<a+b+c || a,b);
9      printf("%d",ret);
10 }

```

问题 输出 调试控制台 终端 端口

ls-1.34.4-win32-x64\debugAdapters\bin\WindowsDebugLauncher.exe' '--stdin=Microsoft-MIEngine-In-ks! ...
y4s50.mep' '--pid=Microsoft-MIEngine-Pid-11odnaqw.y35' '--dbgExe=D:\c\mingw64\bin\gdb.exe' '--int! erpreter=mi'
1
PS D:\c\vscode>

powershell
C/C++: ...
cppdbg: fo...
cppdbg: lo...

行 9, 列 22 空格: 4 UTF-8 CRLF { } C

二、

1. 储存姓名与年龄

在我“恍然大悟”之后, 我决定用 `char name[]`, `int age` 分别储存姓名与年龄 (当然指针也可以储存姓名, 但是一则不能改, 二则如果要使用申请动态内存的方法, 在此处就“言不尽”了, 故不多赘述)

2.原格式打印姓名与年龄

我原本的输入是这样的：

```
scanf("%s %d",name,&age);
```

但考虑到，如果有一个歪果仁，他的名字里有空格（或者太长导致越位）那不就尴尬了，于是我写成了这样：

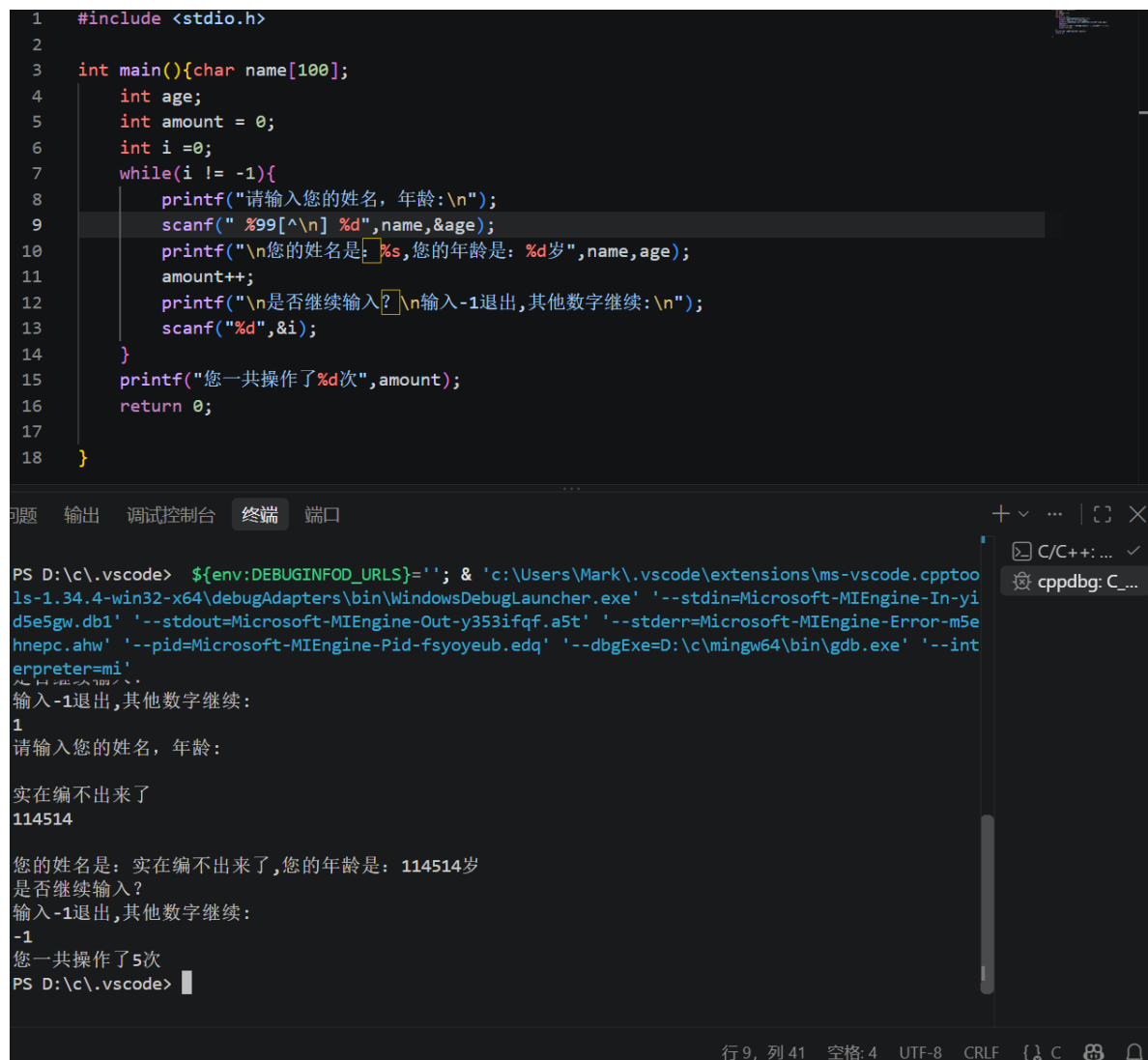
```
scanf("%99[^\n] %d",name,&age);
```

结果在运行的时候出现了问题，因为操作的最后一步“回车”会留在缓冲区，等导致第二次循环第一个字符就是“回车”，没办法再输入。于是我在前面加入了一个空格：

```
scanf( "%99[^\n] %d",name,&age);
```

简直完美！

.....其他功能的添加就不过多赘述，请看VCR Pic:



```
1  #include <stdio.h>
2
3  int main(){char name[100];
4      int age;
5      int amount = 0;
6      int i =0;
7      while(i != -1){
8          printf("请输入您的姓名，年龄:\n");
9          scanf(" %99[^\n] %d",name,&age);
10         printf("\n您的姓名是: %s,您的年龄是: %d岁",name,age);
11         amount++;
12         printf("\n是否继续输入?\n输入-1退出,其他数字继续:\n");
13         scanf("%d",&i);
14     }
15     printf("您一共操作了%d次",amount);
16     return 0;
17
18 }
```

PS D:\c\.\vscode> \${env:DEBUGINFOD_URLS}='' & 'c:\Users\Mark\.vscode\extensions\ms-vscode.cpptools-1.34.4-win32-x64\debugAdapters\bin\WindowsDebugLauncher.exe' '--stdin=Microsoft-MIEngine-In-yidSe5gw.db1' '--stdout=Microsoft-MIEngine-Out-y353ifqf.a5t' '--stderr=Microsoft-MIEngine-Error-m5ehnec.ahw' '--pid=Microsoft-MIEngine-Pid-fsyoyeub.edq' '--dbgExe=D:\mingw64\bin\gdb.exe' '--interpreter=mi'

输入-1退出,其他数字继续:

1

请输入您的姓名，年龄:

实在编不出来了

114514

您的姓名是: 实在编不出来了,您的年龄是: 114514岁

是否继续输入?

输入-1退出,其他数字继续:

-1

您一共操作了5次

PS D:\c\.\vscode>

```
1  #include <stdio.h>
2
3  int main(){char name[100];
4      int age;
5      int amount = 0;
6      int i =0;
7      while(i != -1){
8          printf("请输入您的姓名，年龄:\n");
9          scanf(" %99[^\n] %d",name,&age);
```

```

10     printf("\n您的姓名是: %s,您的年龄是: %d岁",name,age);
11     amount++;
12     printf("\n是否继续输入? \n输入-1退出,其他数字继续:\n");
13     scanf("%d",&i);
14 }
15 printf("您一共操作了%d次",amount);
16 return 0;
17
18 }

```

Part3

1.某人三项

```

1  #include <stdio.h>
2
3  //考虑到求平均数，总成绩时难免会出现小数
4  //于是，将部分数据改成了双精浮点数。
5  double power(int i,int amount);
6  double zh_function(double p,double f);
7  int compare(double zh1,double zh2,double zh3);
8  //观察到在计算综合成绩时依然存在大面积复用情况
9  //于是，除题目要求外，我额外添加了zh_n函数，以期减少主函数中的复用现象。
10 double zh_n(int x,int y,int z);
11
12 int main(){
13     int x1, x2, x3;
14     int y1, y2, y3;
15     int z1, z2, z3;
16
17     printf("请输入小明的三项成绩（顺序为A B C,以一个空格为间隔）：");
18     scanf("%d %d %d", &x1, &x2, &x3);
19     printf("请输入小强的三项成绩（顺序为A B C,以一个空格为间隔）：");
20     scanf("%d %d %d", &y1, &y2, &y3);
21     printf("请输入小林的三项成绩（顺序为A B C,以一个空格为间隔）：");
22     scanf("%d %d %d", &z1, &z2, &z3);
23
24     double zh1 = zh_n(x1,x2,x3);
25     double zh2 = zh_n(y1,y2,y3);
26     double zh3 = zh_n(z1,z2,z3);
27     //将各自的综合成绩计算出来，更直观的同时，还可以判断compare函数有没有出错
28     //但是，这个数值太丑了。。
29     printf("小明的综合成绩是%2f\n",zh1);
30     printf("小强的综合成绩是%2f\n",zh2);
31     printf("小林的综合成绩是%2f\n",zh3);
32
33     compare(zh1,zh2,zh3);
34     return 0;
35 }
36 //虽然计算方差只需要用到平方，但写出真正意义上的乘方函数而不是平方函数，是一劳永逸的事情，
    何乐而不为呢？
37 double power(int i,int amount){
38     double sum = 1;
39     for(int index =0;index<amount;index++){

```

```

40     sum = sum*i;
41 }
42 return sum;
43 }
44 //封装综合成绩的函数，乏善可陈。
45 double zh_function(double p,double f){
46     double sum = 3*p-f/3;
47     return sum;
48 }
49 /*一方面我并不是很想要直接复制题目代码粘贴
50 另一方面，个人认为这样更加符合我的思考过程（看起来稍显复杂，其实也还好吧...嗯..应该吧）
51 另外，我取了等号，这样也许会严谨一些*/
52 int compare(double zh1,double zh2,double zh3){
53     if(zh1 >= zh2){
54         if(zh3 >=zh1){
55             printf("小林 >=小明 >=小强");
56         }else if(zh3 >= zh2){
57             printf("小明 >=小林 >=小强");
58         }else{
59             printf("小明 >=小强 >=小林");
60         }
61     }else{
62         if(zh3 >= zh2){
63             printf("小林 >=小强 >= 小明");
64         }else if(zh3 >= zh1){
65             printf("小强 >=小林 >=小明");
66         }else{
67             printf("小强 >=小明 >= 小林");
68         }
69     }
70 }
71 return 0;
72 }
73 double zh_n(int x,int y,int z){
74     double p = (x + y + z) / 3.0;
75     double f = power((p-x),2)+power((p-y),2)+power((p-z),2);
76     double zh = zh_function(p,f);
77     return zh;
78 }

```

运行结果如下

```
8 //观察到在计算综合成绩时依然存在大面积复用情况
10 double zh_n(int x,int y,int z);
11
12 int main(){
13     int x1, x2, x3;
14     int y1, y2, y3;
15     int z1, z2, z3;
16
17     printf("请输入小明的三项成绩(顺序为A B C,以一个空格为间隔): ");
18     scanf("%d %d %d", &x1, &x2, &x3);
19     printf("请输入小强的三项成绩(顺序为A B C,以一个空格为间隔): ");
20     scanf("%d %d %d", &y1, &y2, &y3);
21     printf("请输入小林的三项成绩(顺序为A B C,以一个空格为间隔): ");
22     scanf("%d %d %d", &z1, &z2, &z3);
23
24     double zh1 = zh_n(x1,x2,x3);
25     double zh2 = zh_n(y1,y2,y3);
26     double zh3 = zh_n(z1,z2,z3);
27     //将各自的综合成绩计算出来,更直观的同时,还可以判断compare函数有没有出错
28     //但是,这个数值太丑了。
29     printf("小明的综合成绩是%2f\n",zh1);
30     printf("小强的综合成绩是%2f\n",zh2);
31     printf("小林的综合成绩是%2f\n",zh3);
32
33     compare(zh1,zh2,zh3);
34     return 0;
35 }
36 //虽然计算成绩只需要用到双精度,但写出来真正的函数而不是双精度函数,且一些函数的重载,何必而不为呢
```

问题 输出 调试控制台 终端 窗口

PS D:\c\vscode> \$(env:DEBUGINFOD_URLS='') & 'c:\Users\Mark\.vscode\extensions\ms-vscode.cpptools-1.34.4-win32-x64\bin\gdb.exe' --interpreter=mi

请输入小明的三项成绩(顺序为A B C,以一个空格为间隔): 87 59 98

请输入小强的三项成绩(顺序为A B C,以一个空格为间隔): 54 65 87

请输入小林的三项成绩(顺序为A B C,以一个空格为间隔): 93 48 69

小明的综合成绩是-11.000000

小强的综合成绩是29.666667

小林的综合成绩是-128.000000

小强 >= 小明 >= 小林

PS D:\c\vscode>

行 31, 列 33 空格: 4 UTF-8 CRLF {} C

2.参数的传递

- 值传递,顾名思义,即实参将自己的数值传递给了形参。值得注意的是,实参与形参除了数值最开始相等以外,实则没有联系。

故而就会出现,swap函数做了无用功,在main之中,a和b可谓稳如泰山,根本不带动的(另外,吐槽一下,main函数里面没有输出,a,b的值就只能利用调试功能查看,差评,必须差评(出题的学长不要记恨我!))

- 要实现真正现实值交换,就需要用到指针来传递a,b的地址(通过指针,使得形参也可以访问a,b的地址,这样形参和实参就建立了联系):

```
1 void swap(int *a, int *b){
2     int temp = *a;
3     *a = *b;
4     *b = temp;
5 }
6
7 int main(){
8     int a = 10;
9     int b = 20;
10    swap(&a, &b);
11
12    printf("a=%d,b=%d",a,b);
13    return 0;//养成好习惯,学长你忘记返回啦!
14 }
```

